

Tutorial Backend - Módulo Categorías

Sistema de votaciones - 7.º Programación

Objetivo del grupo

Implementar el recurso que administra las categorías de votación. Las categorías no deben quedar escritas de forma fija en el código, porque los organizadores todavía pueden definir o modificar cuáles se utilizarán.

Estructura general del backend

```
backend/  
├── src/  
│   ├── config/  
│   │   └── db.js  
│   ├── controllers/  
│   ├── models/  
│   ├── routes/  
│   ├── services/  
│   └── app.js  
├── .env  
├── .env.example  
├── .gitignore  
├── package.json  
└── README.md
```

Cada grupo debe modificar únicamente los archivos correspondientes a su recurso, salvo que sea necesario coordinar un cambio común con el resto del equipo.

Flujo obligatorio

Frontend → Route → Controller → Service → Model → MongoDB

- Route: define la URL y llama al controller.
- Controller: lee req.params, req.query o req.body; llama al service y responde con res.
- Service: contiene validaciones y lógica del recurso.
- Model: define el esquema de Mongoose y realiza las operaciones sobre MongoDB.
- El controller no consulta modelos directamente.
- El service no utiliza req ni res.

Archivos asignados

- src/models/Categoria.js
- src/services/categoriasService.js
- src/controllers/categoriasController.js
- src/routes/categoriasRoutes.js

Convenciones que deben respetarse

- Todo el código y los nombres propios del proyecto estarán en español.
- Variables y funciones: camelCase.
- Modelos: singular y PascalCase, por ejemplo Grupo.js.
- Controllers: plural + Controller, por ejemplo gruposController.js.
- Routes: plural + Routes, por ejemplo gruposRoutes.js.
- Services: plural + Service, por ejemplo gruposService.js.
- URLs: minúsculas y sustantivos en plural.
- Usar const por defecto, let solo cuando el valor cambie.
- Comillas simples, punto y coma e indentación de dos espacios.
- Usar async/await para operaciones asíncronas.
- Exportar con module.exports.
- No guardar claves ni conexiones privadas dentro del código.

Modelo de datos inicial

Los campos siguientes son una base de trabajo. Si durante el desarrollo se necesita agregar o cambiar un campo, debe acordarse antes con el equipo para no romper la integración.

- **nombre:** Nombre visible de la categoría, por ejemplo Favorito del público.
- **descripcion:** Explicación breve de qué se vota en esa categoría.
- **activa:** Permite habilitar o deshabilitar una categoría sin eliminarla; puede dejarse como true por defecto.

Métodos que debe implementar el grupo

Método	Responsabilidad
obtenerCategorias	Obtener todas las categorías.
obtenerCategoriaPorId	Obtener una categoría por su ID.
crearCategoria	Registrar una categoría.
actualizarCategoria	Modificar una categoría.
eliminarCategoria	Eliminar una categoría.

Rutas que deben quedar disponibles

HTTP	Ruta	Controller
GET	/api/categorias	obtenerCategorias
GET	/api/categorias/:id	obtenerCategoriaPorId
POST	/api/categorias	crearCategoria
PUT	/api/categorias/:id	actualizarCategoria
DELETE	/api/categorias/:id	eliminarCategoria

Orden recomendado de trabajo

Paso	Archivo	Qué debe hacerse
1	models/Categoria.js	Definir el esquema y modelo.
2	services/categoriasService.js	Implementar CRUD con los métodos estándar de Mongoose.
3	controllers/categoriasController.js	Preparar respuestas HTTP y manejar recursos inexistentes.

4	routes/categoriasRoutes.js	Crear las cinco rutas del recurso.
5	src/app.js	Coordinar el registro de categoriasRoutes con el responsable general.

Ejemplos base

Estos fragmentos sirven como referencia de estructura. Deben adaptarse a los campos definitivos y a las validaciones que acuerde el equipo.

Modelo

```
const mongoose = require('mongoose');

const categoriaSchema = new mongoose.Schema({
  nombre: { type: String, required: true, trim: true },
  descripcion: { type: String, default: " " },
  activa: { type: Boolean, default: true }
}, { timestamps: true });

module.exports = mongoose.model('Categoria', categoriaSchema);
```

Service

```
const Categoria = require('../models/Categoria');

async function obtenerCategorias() {
  return Categoria.find();
}

async function crearCategoria(datosCategoria) {
  return Categoria.create(datosCategoria);
}

module.exports = {
  obtenerCategorias,
  crearCategoria
};
```

Controller

```
const categoriasService = require('../services/categoriasService');

async function obtenerCategorias(req, res, next) {
  try {
    const categorias = await categoriasService.obtenerCategorias();
    return res.status(200).json(categorias);
  } catch (error) {
    return next(error);
  }
}
```

```
module.exports = { obtenerCategorias };
```

Routes

```
const express = require('express');  
const categoriasController = require('../controllers/categoriasController');
```

```
const router = express.Router();
```

```
router.get('/', categoriasController.obtenerCategorias);  
router.get('/:id', categoriasController.obtenerCategoriaPorId);  
router.post('/', categoriasController.crearCategoria);  
router.put('/:id', categoriasController.actualizarCategoria);  
router.delete('/:id', categoriasController.eliminarCategoria);
```

```
module.exports = router;
```

Pruebas mínimas antes de entregar

- GET /api/categorias devuelve todas las categorías.
- GET por ID controla categorías inexistentes.
- POST no permite crear una categoría sin nombre.
- PUT actualiza correctamente.
- DELETE elimina correctamente.
- No escribir categorías fijas dentro de routes, controllers o frontend.

Criterio de finalización

La parte asignada se considera lista cuando sus rutas responden correctamente, los errores básicos están controlados, los nombres coinciden con este documento y no se modificaron archivos pertenecientes a otros módulos sin coordinación.